

Complemento Práctico: Ejercicio Integrador

Arquitectura de Colecciones

En este documento final, enfrentamos un desafío de diseño completo que representa la culminación del aprendizaje sobre colecciones en Java. El enunciado nos presenta el caso de una empresa que distribuye instrumentos musicales a través de múltiples sucursales, un problema clásico en programación orientada a objetos que involucra **Colecciones Anidadas**: una Fábrica contiene una lista de Sucursales, y cada Sucursal mantiene su propia lista de Instrumentos.

Este ejercicio integrador nos permitirá analizar las decisiones de diseño más importantes, comprender cómo delegar responsabilidades correctamente para evitar el antipatrón de la "Clase Dios" que concentra toda la lógica, y resolver algoritmos matemáticos sobre colecciones incluyendo el cálculo de porcentajes con precisión decimal.



Contenido

Arquitectura

Modelado de la cadena de mando: Fábrica → Sucursal → Instrumento

Enumeraciones

Categorizando instrumentos con TipoInstrumento para evitar strings mágicos

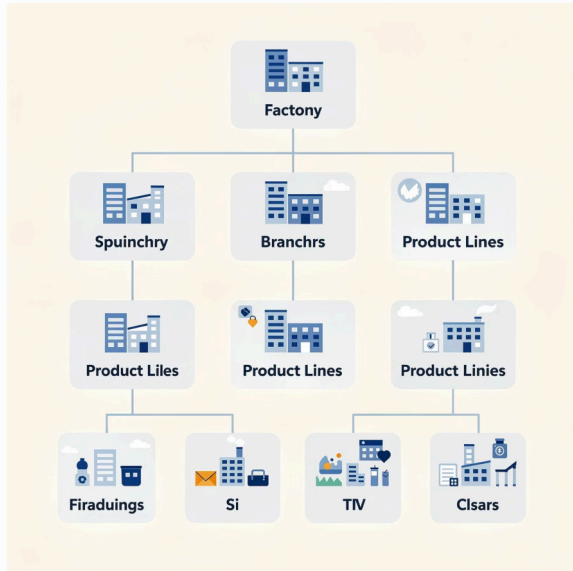
Algoritmos

Búsqueda, borrado por ID y cálculo de porcentajes con casting

Buenas Prácticas

El principio de delegación en estructuras de listas anidadas

La Jerarquía de Clases



El primer paso fundamental en cualquier diseño orientado a objetos es identificar correctamente los sustantivos del enunciado, ya que estos se convertirán en nuestras clases principales. En este caso, tres entidades emergen claramente del análisis del dominio del problema.

Fábrica: La entidad principal y coordinadora de todo el sistema. Es la propietaria de la estructura completa y el punto de entrada para operaciones globales.

Sucursal: La unidad operativa intermedia. Cada sucursal tiene un nombre identificador y, lo más importante, mantiene su propio stock independiente de instrumentos.

Instrumento: El producto final. Cada instrumento posee un identificador único, un precio y una categoría representada por su tipo.

Estructura de Datos: Respetando la Jerarquía

La estructura de datos debe reflejar fielmente la jerarquía del mundo real. Este es un principio fundamental en el diseño orientado a objetos: **el código debe modelar la realidad del dominio.**

❏ Clase Fábrica

```
public class Fabrica {  
    private ArrayList<Sucursal> sucursales;  
    // La fábrica NO tiene instrumentos sueltos  
    // Tiene sucursales que gestionan instrumentos  
}
```

❏ Clase Sucursal

```
public class Sucursal {  
    private ArrayList<Instrumento> instrumentos;  
    // La sucursal es la dueña real de los instrumentos  
    // y conoce sus operaciones específicas  
}
```

❏ Clase Instrumento

```
public class Instrumento {  
    private String id;  
    private double precio;  
    private TipoInstrumento tipo; // Enum  
}
```

Lección crítica: Respetar esta jerarquía es vital para mantener el código organizado y mantenible. La Fábrica no debería acceder o manipular directamente la lista de instrumentos; en su lugar, debe comunicarse con la Sucursal correspondiente y delegarle esa responsabilidad. Este patrón de delegación es fundamental para evitar el acoplamiento excesivo.

Enumeraciones: Adiós a los Strings Mágicos

El Problema

Usar String para tipos permite inconsistencias: "Percusión", "percusion", "PERCUSION" serían valores diferentes

La Solución: Enum

El enunciado menciona tres categorías de instrumentos: "Percusión", "Viento" y "Cuerda". En lugar de representar estos tipos como cadenas de texto (String), implementamos un tipo enumerado que garantiza seguridad de tipos.

```
public enum TipoInstrumento {  
    PERCUSION,  
    VIENTO,  
    CUERDA;  
}
```

Unicidad Garantizada

Solo existen los valores definidos, imposible crear variantes incorrectas

Comparación Segura

Podemos usar el operador == para comparar valores del enum de forma confiable

Autocompletado en IDE

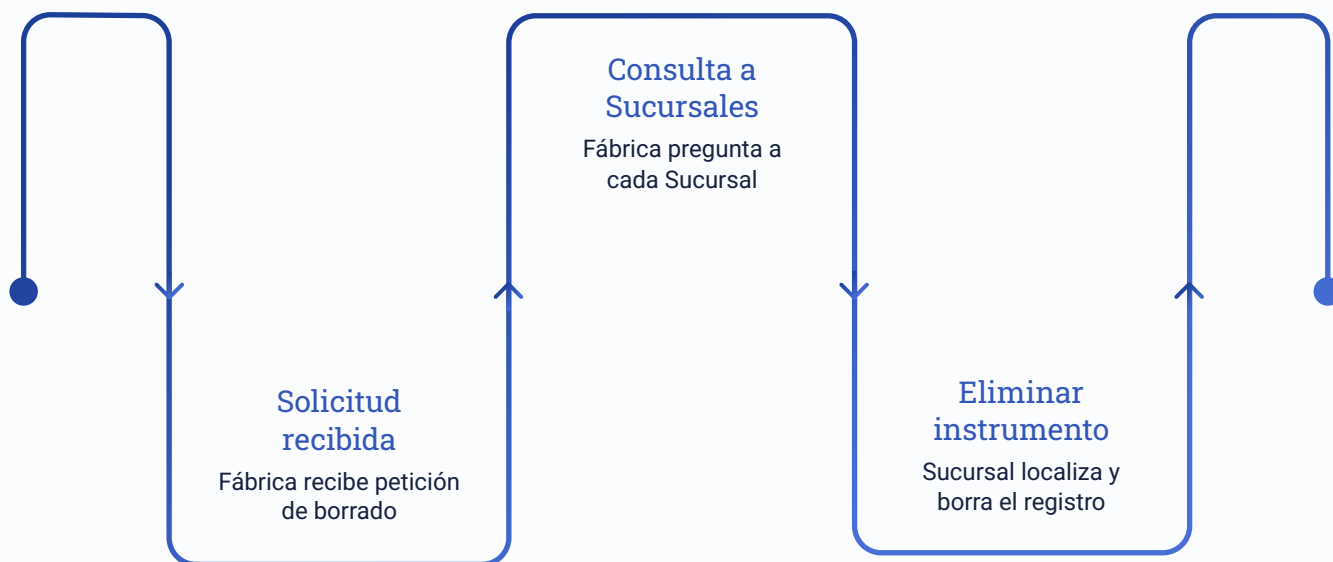
Los entornos de desarrollo sugieren automáticamente los valores válidos

Al calcular porcentajes por tipo de instrumento, no corremos ningún riesgo de que las comparaciones fallen debido a diferencias en mayúsculas, minúsculas o espacios adicionales. El Enum garantiza consistencia absoluta en todo el sistema.

Algoritmo de Borrado: El Desafío de la Delegación

El método de borrado por identificador representa uno de los algoritmos más interesantes del ejercicio. El desafío principal radica en que **no sabemos de antemano en qué sucursal se encuentra el instrumento** que deseamos eliminar. Este problema requiere una estrategia de búsqueda coordinada entre múltiples niveles de la jerarquía.

La solución correcta implementa el **patrón de delegación en cascada**. La Fábrica actúa como coordinadora general, mientras que cada Sucursal es responsable de gestionar su propio inventario. Esta separación de responsabilidades es crucial para mantener un código limpio y mantenible.



La implementación utiliza un enfoque de búsqueda secuencial con terminación temprana: tan pronto como una sucursal confirma que ha encontrado y eliminado el instrumento, el proceso completo finaliza, evitando búsquedas innecesarias en las sucursales restantes.

Implementación del Borrado en Dos Niveles

Nivel Sucursal: El Ejecutor

La sucursal conoce íntimamente su inventario y puede realizar la operación de eliminación de forma eficiente:

```
public Instrumento
borrarInstrumento(String id) {
    // Reutilizamos el método de
    búsqueda
    Instrumento aBorrar =
    this.buscarPorId(id);

    if (aBorrar != null) {
        this.instrumentos.remove(aBorrar);
    }

    // Devolvemos el instrumento borrado
    // o null si no existía
    return aBorrar;
}
```

Este método encapsula la lógica de búsqueda y eliminación, retornando el instrumento eliminado como confirmación.

Nivel Fábrica: El Coordinador

La fábrica orquesta la búsqueda delegando a cada sucursal:

```
public void borrarInstrumento(String id) {
    for (Sucursal suc : this.sucursales) {
        Instrumento borrado =
            suc.borrarInstrumento(id);

        if (borrado != null) {
            System.out.println(
                "Instrumento borrado de: "
                + suc.getNombre()
            );
            return; // Terminación temprana
        }
    }
    System.out.println(
        "Instrumento no encontrado"
    );
}
```

La fábrica nunca accede directamente a las listas de instrumentos, respetando el encapsulamiento.

Cálculo de Porcentajes: Matemáticas con Colecciones

Uno de los requerimientos más interesantes del ejercicio es calcular el **porcentaje de instrumentos por tipo en una sucursal determinada**. La fórmula matemática es simple: $(\text{Cantidad del Tipo X} / \text{Total de Instrumentos}) \times 100$. Sin embargo, la implementación en Java requiere atención especial a un detalle crucial.



La Trampa de División Entera

Si tenemos 2 instrumentos de viento y 5 totales, la expresión $2 / 5$ en Java produce 0, no 0.4



La Solución: Casting

Debemos convertir al menos uno de los operandos a double usando (double) antes de la división



Resultado Correcto

Con casting obtenemos decimales precisos: $(\text{double})2 / 5 \times 100 = 40.0\%$

Este es un error común en programación que puede causar resultados completamente incorrectos. Java realiza división entera cuando ambos operandos son enteros, truncando cualquier parte decimal del resultado. El casting a double fuerza la división en punto flotante, preservando la precisión necesaria para cálculos de porcentajes.

Implementación Completa del Cálculo de Porcentajes

La implementación del método de cálculo de porcentajes requiere varios pasos cuidadosamente coordinados. Primero debemos localizar la sucursal específica, luego contar los instrumentos del tipo solicitado, y finalmente realizar el cálculo con la precisión decimal correcta.

❏ Código Completo con Manejo de Casos Especiales

```
public double porInstrumentosPorTipo(
    String nombreSucursal,
    TipoInstrumento tipo
){
    // Paso 1: Buscar la sucursal
    Sucursal suc = buscarSucursal(nombreSucursal);
    if (suc == null) return 0; // Sucursal no existe

    // Paso 2: Obtener totales
    int total = suc.getInstrumentos().size();
    int cantidadTipo = 0;

    // Paso 3: Contar instrumentos del tipo
    for (Instrumento inst : suc.getInstrumentos()) {
        if (inst.getTipo() == tipo) {
            cantidadTipo++;
        }
    }

    // Paso 4: Validar división por cero
    if (total == 0) return 0;

    // Paso 5: CLAVE - Casting para decimales
    return (double) cantidadTipo / total * 100;
}
```

01

Localización

Buscar la sucursal por nombre y validar su existencia

02

Conteo

Recorrer instrumentos comparando tipos con el enum

03

Validación

Prevenir división por cero si la sucursal está vacía

04

Cálculo

Aplicar casting y multiplicar por 100 para obtener porcentaje

Conclusión: Cierre de la Unidad de Colecciones

Con este ejercicio integrador, hemos demostrado que programar profesionalmente no consiste únicamente en conocer la sintaxis de ArrayList o las operaciones básicas de colecciones. El verdadero desafío está en **saber dónde ubicar cada pieza de lógica** y cómo organizar las responsabilidades entre las diferentes clases del sistema.

Sucursal: La Experta Local

Conoce íntimamente sus instrumentos y sabe cómo contarlos, buscarlos, borrarlos y listarlos eficientemente

Fábrica: La Coordinadora Global

Gestiona todas las sucursales y deriva pedidos a la sucursal correcta sin violar el encapsulamiento

Main: El Punto de Entrada

Solo inicia la Fábrica, carga datos de prueba y ejecuta operaciones de alto nivel

La arquitectura resultante es escalable, mantenible y respeta los principios fundamentales de la programación orientada a objetos. Hemos aprendido a trabajar con colecciones anidadas, implementar delegación correcta, usar enumeraciones para categorización segura, y manejar operaciones matemáticas con la precisión adecuada.

¡Felicitaciones! Has completado exitosamente el módulo de Colecciones. Ahora posees las habilidades necesarias para manipular grandes volúmenes de datos en memoria utilizando arquitectura profesional. Estos conceptos son fundamentales para tu desarrollo como programador y te preparan para enfrentar desafíos más complejos en tu carrera profesional.